CSE/DSC 234 Project 1: RAG and Context Engineering

Anonymous Author(s)
Affiliation
Address
email

Abstract

This report describes a retrieval-augmented question answering system for the RapidFire AI documentation corpus [1]. The final pipeline is a deterministic TF-IDF RAG system with line-aware chunking, lightweight reranking, targeted related-source expansion, and prompt-token budget control. The provided validation set was used as a development and evaluation set, while the submitted golden Q&A file was curated separately from the documentation corpus.

1 Data Creation Methodology

The submitted golden data file is $\mathrm{data/golden_qa.json}$. I first manually searched the RapidFire AI documentation file by file and wrote 40 candidate Q&A pairs with source locations. During this pass, I roughly identified the line ranges that could serve as supporting material for each answer. After that, I made a second pass for deduplication and coverage, removing repeated or overly similar questions and keeping 25 higher-quality pairs. The released $\mathrm{data/validation\text{-}set\text{-}golden\text{-}qa\text{-}pairs.json}$ file was treated only as the provided validation/dev set for evaluation and tuning, not as the source of the submitted golden data.

The manual search focused on questions that could be answered directly from the corpus. I targeted four types: factual lookup, conceptual explanation, procedural usage, and comparative reasoning. The final set covers different documentation areas, including multi-config generation, experiment control, RAG specs, generator configs, online aggregation, dashboards, and glossary terms.

Each golden item contains a question id, a question, a gold answer, and one or more source spans. I checked each span against the corresponding $\mathrm{.rst}$ file so that the answer was supported by the cited lines. I did not use the validation answers as submitted golden answers. No extra custom judge metrics were added beyond the released retrieval and generation metrics.

2 Final Pipeline Architecture

The final pipeline uses the TC02 setting from the token-budget experiments: 420-word chunks, 80-word overlap, one primary retrieved chunk, title and symbol reranking, up to five related source files, three-line source/context expansion, and a 1900-token prompt cap. The script writes the answer, source spans, and the retrieved context that was actually passed to the generator. The source spans are kept consistent with the serialized context.

Document loading and splitting. The script directly loads $\mathrm{.rst}$ files from the supplied corpus directory. This corresponds to RapidFire AI's document-loader stage, but keeps the final autograder entry point standalone. During loading, the code keeps every file name, raw line list, document title, and section heading. Section headings are detected from reStructuredText underline patterns. Blank

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

lines are skipped for chunking, but line numbers are preserved for evidence reporting. Each chunk is built from about 420 words, and adjacent chunks overlap by about 80 words. The chunk text is lightly cleaned by removing common reStructuredText markup such as inline code roles and image directives.

Embedding and retrieval. The RapidFire dense experiments used OpenAI-compatible embeddings with FAISS vector stores [4]. The final pipeline keeps the same RAG structure, but replaces the dense embedding and vector store with a local TF-IDF module [2]. Each indexed document is the concatenation of file name, section heading, and cleaned chunk text. The TF-IDF vectorizer uses lowercasing, Unicode accent stripping, word-level analysis, unigram and bigram features, sublinear term frequency, $\mathrm{max_df}{=}0.92$, and $\mathrm{min_df}{=}1$. Retrieval computes cosine similarity between the question vector and every chunk vector:

$$s_{\text{tfidf}}(q, c) = \frac{v_q \cdot v_c}{\|v_q\|\|v_c\|}.$$

This choice was faster and more stable than dense retrieval on the small technical corpus. The implementation uses scikit-learn's TF-IDF and cosine-similarity utilities [3].

Reranking and related context. After TF-IDF retrieval, I apply a lightweight reranking score. Let T_q be the non-stopword lexical terms in the question, T_c be title/section terms for a chunk, and A_q be API-like symbols in the question. The final retrieval score is:

$$s(q, c) = s_{\text{tfidf}}(q, c) + 0.12\frac{|T_q \cap T_c|}{|T_q|} + 0.02 \sum_{a \in A_q} \mathbf{1}[a \in c].$$

For API-symbol questions, glossary chunks receive a small penalty when they match only broad terminology. After reranking, the pipeline keeps the best chunk and then adds targeted related files for a few patterns: tutorial use cases, vector-store questions, glossary/terminology questions, and comparison questions. This related-source step addresses the limitation of single-chunk retrieval: some RapidFire answers are split across overview pages and concrete tutorial files.

Context budget and generation. The context builder serializes each selected source as a labeled block with file name, line range, section, score, core chunk lines, and three surrounding lines. Before adding a block, the script checks the full generator prompt, not only the context text. When an API key is available, this check calls the generation model with $\mathrm{max_tokens}{=}1$ and reads the real prompt-token count from $\mathrm{response.usage.prompt_tokens}$. If adding the next block would exceed the 1900-token cap, the script trims only that final block by splitting it into paragraphs or sentences and binary-searching for the largest prefix that still fits. The 1900 cap leaves margin under the 2000-token project limit for tokenizer variance and prompt formatting. Generation uses the requested gateway model, with an extractive fallback only when the API call is unavailable. The required packages are numpy, $\mathrm{scikit\text{-}learn}$, and openai.

3 Experimentation Methodology

I started from the RapidFire AI tutorial workflow and wrote an initial experiment driver using RapidFire's own RAG components: document loading, chunking, embedding configuration, FAISS vector store options, search settings, reranking, generation, and evaluation. I first varied the standard RapidFire-style knobs such as chunk size, overlap, similarity vs. MMR search, retrieval depth, and CrossEncoder reranker depth. These dense runs are the D-series rows in Table 1.

After that baseline, I kept the same overall RAG architecture but treated retrieval as a plug-and-play module. Because this corpus is a small technical documentation set with many exact API names and section titles, I tested a TF-IDF replacement for dense embedding retrieval. The first TF-IDF runs already improved retrieval substantially, especially with the 420/80 chunk setting. I then added two targeted improvements: title/API-symbol boosting and related-source expansion. The limitation of plain TF-IDF was that it could retrieve the right local chunk but miss a nearby overview page, tutorial page, or comparison page. The related-source rules were designed to handle those cases without blindly adding many chunks.

Table 1: Main RapidFire validation configurations. Scores are from the released validation/dev set.

ID	Config name	Retrieval	Generation	Overall	Notes
D01	dense 320/64 similarity k5 rerank3	**0.6499**	0.8267	0.7383	FAISS dense
D02	dense 320/64 similarity k5 rerank5	0.5934	0.8830	0.7382	FAISS dense
D03	dense 320/64 similarity k8 rerank3	0.6324	0.8178	0.7251	FAISS dense
D04	dense 320/64 similarity k8 rerank5	0.5823	0.8356	0.7089	FAISS dense
D05	dense 320/64 MMR k8 fetch20 rerank3	0.6462	0.8444	**0.7453**	FAISS MMR
D06	dense 320/64 MMR k8 fetch20 rerank5	0.5865	0.8207	0.7036	FAISS MMR
D07	dense 512/96 similarity k5 rerank3	0.6106	0.8370	0.7238	FAISS dense
D08	dense 512/96 similarity k5 rerank5	0.5668	**0.8859**	0.7264	FAISS dense
D09	dense 512/96 similarity k8 rerank3	0.6153	0.8578	0.7366	FAISS dense
D10	dense 512/96 similarity k8 rerank5	0.5579	0.8533	0.7056	FAISS dense
D11	dense 512/96 MMR k8 fetch20 rerank3	0.6153	0.8741	0.7447	FAISS MMR
D12	dense 512/96 MMR k8 fetch20 rerank5	0.5579	0.8533	0.7056	FAISS MMR
T01	tfidf 320/64 k1 no title no related	0.7370	0.8533	0.7952	baseline small chunks
T02	tfidf 420/80 k1 no title no related	0.8198	0.8444	0.8321	baseline TF-IDF
T03	tfidf 512/96 k1 no title no related	0.7975	0.8474	0.8225	larger chunks
T04	tfidf 320/64 k1 title no related	0.8037	0.8681	0.8359	title boost
T05	tfidf 420/80 k1 title no related	0.9148	0.8933	0.9041	title boost
T06	tfidf 512/96 k1 title no related	0.8704	0.8474	0.8589	title boost
T07	tfidf 320/64 k1 title related	0.8310	0.8770	0.8540	related docs
T08	tfidf 420/80 k1 title related	**0.9421**	0.9156	0.9288	related docs
T09	tfidf 512/96 k1 title related	0.8977	0.9067	0.9022	related docs
T10	tfidf 420/80 k1 title related expand5	**0.9421**	**0.9437**	**0.9429**	best early TF-IDF
T11	tfidf 420/80 k1 title related expand10	**0.9421**	0.9274	0.9348	more context
T12	tfidf 420/80 k3 title related expand5	**0.9421**	0.9200	0.9311	larger top-k
T13	tfidf 420/80 k5 title related expand5	**0.9421**	0.9274	0.9348	larger top-k

Table 2: Token-budget ablation. All prompt-token counts are measured from the generator API. Bold marks the best metric values; for max prompt tokens, bold marks the smallest prompt.

ID	Main knobs	Retrieval	Generation	Overall	Max prompt tokens
TC01	420/80, related 5, cap 1950	**0.9389**	0.9422	0.9406	1909
TC02	420/80, related 5, cap 1900	**0.9389**	**0.9496**	**0.9443**	1858
TC03	384/64, related 5, cap 1900	0.8199	0.8681	0.8440	1895
TC04	320/64, related 5, cap 1900	0.8310	0.8711	0.8511	1891
TC05	256/48, related 5, cap 1900	0.7755	0.8148	0.7951	1880
TC06	420/80, related 4, cap 1900	**0.9389**	0.9304	0.9346	1858
TC07	420/80, related 3, cap 1900	0.9351	0.9481	0.9416	**1744**
TC08	320/64, balanced packing	0.8310	0.8726	0.8518	1891
TC09	320/64, question-aware packing	0.8310	0.8326	0.8318	1900
TC10	256/48, staged packing	0.7755	0.8074	0.7914	1892

Finally, I found that the earlier experiments did not explicitly verify the full generator prompt against the 2000-token budget. I therefore ran the TC-series token-budget ablation in Table 2. These runs measured real prompt tokens from the generation model API and tested several context-packing choices. TC02 had the best validation overall, so I adopted it as the final setting in `main.py`. All tables below report validation/dev-set performance only.

After the main RapidFire comparison, I ran a token-budget ablation that measured real prompt token counts rather than local estimates. This tested ten variants around the best TF-IDF design by changing chunk size, related-source count, context-packing strategy, and prompt-token cap.

4 Results and Analysis

The main conclusion is that lexical retrieval worked better than dense retrieval for this small technical documentation corpus. Dense FAISS configurations had generation scores that were often reasonable, but retrieval scores were much lower. The best dense setting was D05 with overall 0.7453, while the best early TF-IDF setting T10 reached 0.9429.

The biggest improvement came from title-aware TF-IDF retrieval. Comparing T02 to T05, the same 420/80 chunking improved retrieval from 0.8198 to 0.9148. Adding related-source expansion improved the same family further: T08 reached 0.9421 retrieval. This suggests that exact technical terms, file titles, and section names are highly predictive in the RapidFire documentation.

More retrieved chunks did not help as much as expected. T12 and T13 used top-k values of 3 and 5, but their generation scores were below T10. The extra context sometimes introduced unnecessary information and made the generator less complete or less focused. Similarly, expanding context to 10 lines in T11 reduced generation relative to T10. The best results came from retrieving concise evidence and adding only targeted related sources.

The token-budget experiments changed the final setting. TC02 kept the same 420/80 chunking and related-source idea, but used real prompt-token checks with a 1900-token cap. This avoided relying on rough local estimates and kept all tested prompts under the 2000-token limit. The current regenerated `output.json` was checked with the released metrics and scored 0.9389 retrieval, 0.9422 partial generation, and 0.9406 partial overall. The small difference from the TC02 ablation generation score is expected because generation and LLM judging can vary slightly across reruns.

The main remaining errors are concentrated in a few validation questions. Retrieval missed or partially matched Q6, Q8, Q22, Q29, and Q38. Generation completeness was also lower on questions where the retrieved context contained the right file but not every detail the reference answer expected. With more time, I would test a hybrid retrieval score combining TF-IDF with dense embeddings, add section-aware query expansion, and tune the related-source rules using more curated examples rather than only the provided validation set.

5 AI Tools Statement

GPT was used to organize experiment commands, summarize metric JSON outputs, polish the report language and logical flow, and provide an overview of the project objective and file structure. The reported results come from the project files and local metric outputs generated by the submitted code and evaluation scripts.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv:2005.11401*, 2020.

[2] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[3] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.